浙江大学

计算机体系结构
实验报告

实验名称 ：	Lab1
姓名 ：	黄予恒
学号 ：	3240102750
学院/专业 ：	计算机学院 人工智能
实验地点 ：	东4-511
实验日期 ：	2026年 3 月 9 日
指导老师 ：	卜凯

1. Objectives and Requirements

Objectives:

- Finish the design of pipelined RV32I CPU
- Implement Pipeline Forwarding Detection and bypass unit
- Master the methods of 1-cycle stall and Predict-not-taken flush design

Requirements

- Understand RISC-V RV32I Instructions
- Understand how forwarding and stall works

2. Contents and Principles

First, there are four files we should complete this time:

- `CtrlUnit.v`
- `cmp_32.v`
- `RV32core.v`
- `HazardDetectionUnit.v`

Next I'll introduce the design of these components one by one.

For the `CtrlUnit.v`, we are asked to assign digital signal to certain wires.

```
...
wire BEQ = Bop && funct3_0;                          //to fill sth. in
wire BNE = Bop && funct3_1;                          //to fill sth. in
wire BLT = Bop && funct3_4;                          //to fill sth. in
...
```

These signals are for decoding the instruction, we only need to use the opcode and funct3 to implement the decoding.

Note that most of the instructions in the same type have the same opcode, but note that `LUI, AUIPC, JAL, JALR` all have **unique opcode**, so we can directly use the opcode to decode them.

For the signal `Branch`, I assign it `JAL | JALR | (B_valid && cmp_res)` to make sure that the branch signal is pulled up to one when unconditional jump or branch occurs.

For the `cmp_ctrl` signal, I extend the former decoded signal to work as **mask codes** to simplify the code implementation :

```
assign cmp_ctrl = {3{BEQ}} & cmp_EQ |
                  {3{BNE}} & cmp_NE | ...
```

Also note that the `cmp_EQ...` signals are set to the same as the implementation in `cmp_32.v`

Next are the ALUSrc control signal. The control signal here must be aligned with our wiring in the `RV32core.v`

Since for the first mux, I0 connects PC and I1 connects rs1_data, our ALUSrc_A should output 1 most of the time, only when `JALR, JAL, AUIPC` happens, should we connect PC to the ALU.

The design of ALUSrcB is the same.

Then for the `rs1use` and `rs2use` signals, these two signals represent whether the register rs1 or rs2 is used, to determine in the HazardDetectionUnit whether we need to forward.

Since the instructions that don't use rs1 are rare, so we assign rs1use as `!(LUI | AUIPC | JAL)`. For the rs2use, we assign it `(R_valid | S_valid | B_valid)` since these three types of instructions use rs2.

For the last signal `hazard_optype`, it is used to represent the state of current instruction, which we could use in the HazardDetectionUnit to determine the type of the current hazard(According to the hazard_optype in different pipeline stages). We use the mask of decoded instructions to assign hazard_optype the correct signal.

For the `cmp_32.v` part, we only need to design a simple comparator.

we simply assign the output signal c as

`assign c = ((res_EQ && EQ)) || ...`, also res_XX as the potential result and XX as the mask code.

For the `RV32core.v` part, we connect all the wires

The `mux_IF` is for selecting the next PC for our PC register, so when the Branch_ctrl is pulled up, then we should feed the jump_PC back, so it should be connected to I1, and connect PC_4_IF to I0 as the default next PC.

The design of mux_forward should be consistent with the design in HazardDetectionUnit, aiming to pass the correct value to forward I will introduce the design of the signals in the next part.

For the mux_A_exe and mux_B_EXE, they are used to choose the input of ALU, the signals are already introduced in the CtrlUnit part.

For the mux_forward_EXE, it is specifically to solve the load-store hazard, which forward the data from the MEM stage directly to EXE stage.

For the `HazardDetectionUnit` part, it is used to detect the type of current hazard and assign the correct signals to the forwarding unit, and also handle the flush when branch happens.

First we need to assign all register's enable signal to 1, to make sure they work properly. Then we design the update of hazard_optype signals of different pipeline registers, and we should mask them with the flush signals.

Load-Use is the only type of hazard that we need to stall, so the stall signal is pulled up when load was detected in EXE stage, non-store is detected in ID stage and the source register of ID stage equals to the target register in the EXE stage.

The forwarding signals must correspond to our connecting in RV32Core.v. if it is 1, then it is EXE ALU Ouput->ID input, if it is 2, then it is MEM ALU->ID input, if it is 3, then it is DMEM output->ID input. Note that all the handling here is based on forwarding back to ID stage.

Then we use the mask to assign the forward_ctrl_X with the correct signal.

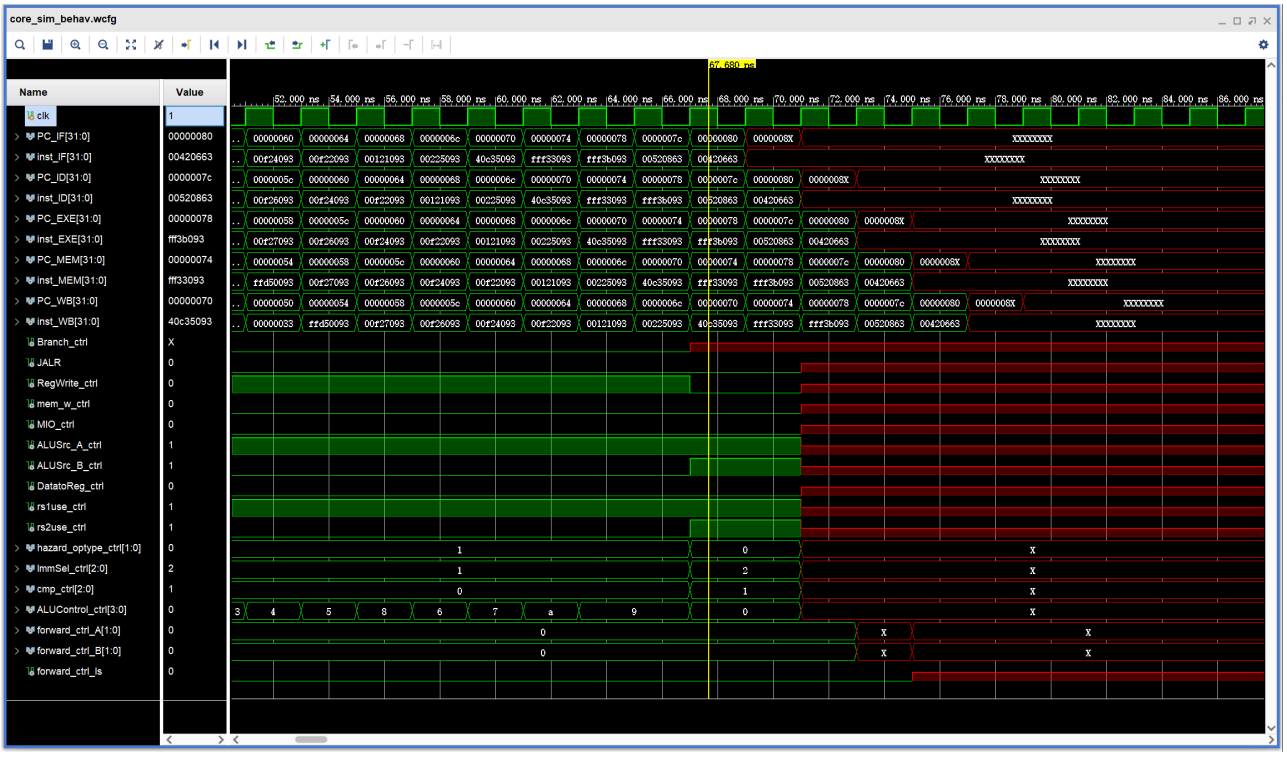
Then for the forward_ctrl_ls, it is specifically for the load-store forwarding, and we only need to examine the signals in the EXE and MEM stage, the logic is the same.

For the flush signals, we first need to have the general stall signal which is from the OR operation of rs1_stall and rs2_stall. When we have the general stall signal, we can control the pipeline registers. When general stall happens, we need to stop the IF update, so PC_EN_IF should be set to !general_stall. The reg_FD_stall should be synchronized with general stall, and the following IDEXE register should be flushed.

When branch happens, we need to flush the IFID register to get the new instruction, so it is set to be synchronized with Branch_ID.

3. Experiment Process

I will introduce how I debug in the whole process.



First, I discover that after the first branch instruction, the PC we get went wrong.

Then I discover a stupid typo in the CtrlUnit.v, so the Branch Signal was incorrect, so the PC is unable to update correctly.

Then I discover that the `forward_ctrl_X` signal displays `X` when it is supposed to be 3, indicating that we are unable to determine the value of `forward_ctrl_X`.

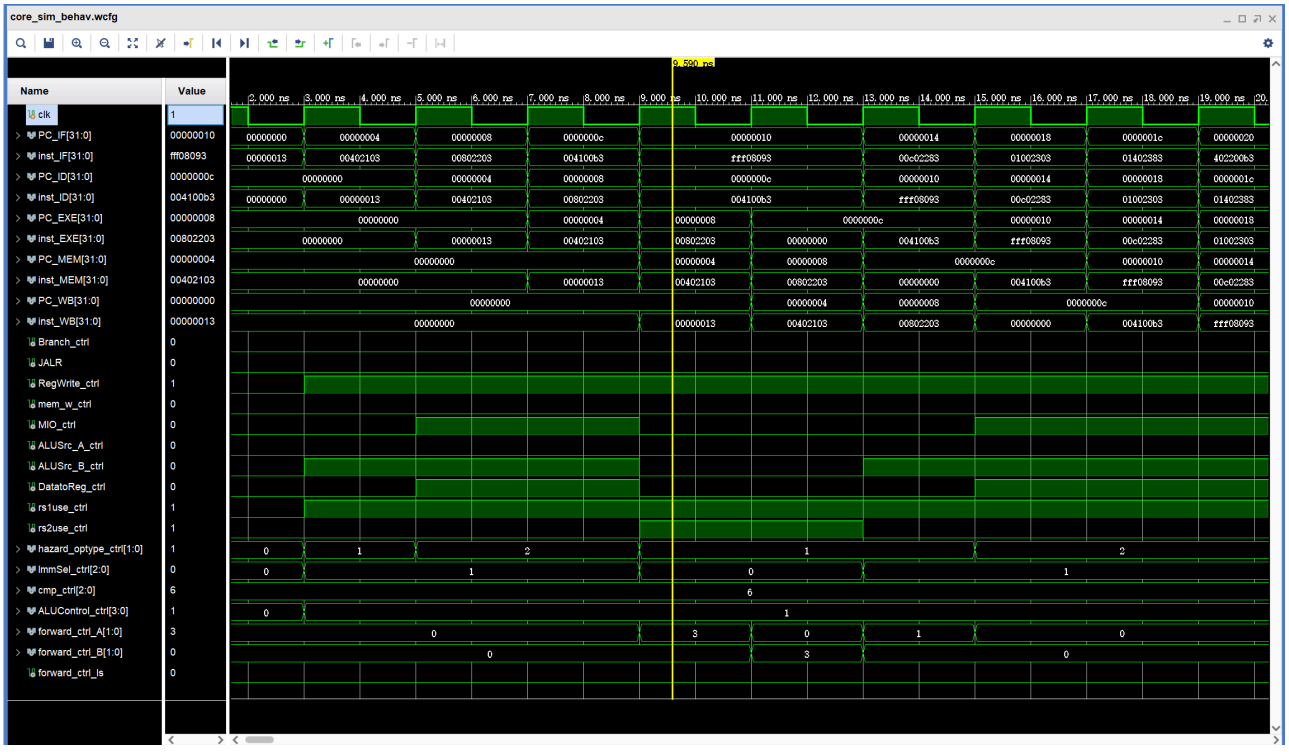
```
always @(posedge clk) begin
    hazard_optype_MEM <= hazard_optype_EXE & {2{~reg_EM_flush}};
    hazard_optype_EXE <= hazard_optype_ID & {2{~reg_DE_flush}};
    // Consider the factor of register flush
end
```

Then I discover that the update process here depends on the `reg_flush` signals, and the signal `reg_EM_flush` was not initialized and not assigned, so the `hazard_optype_MEM` can't be correctly updateted, making it unable to get the value of `hazard_optype_MEM`, so I initialized it with `1'b0`, and the problem was solved.

4. Analysis and Results

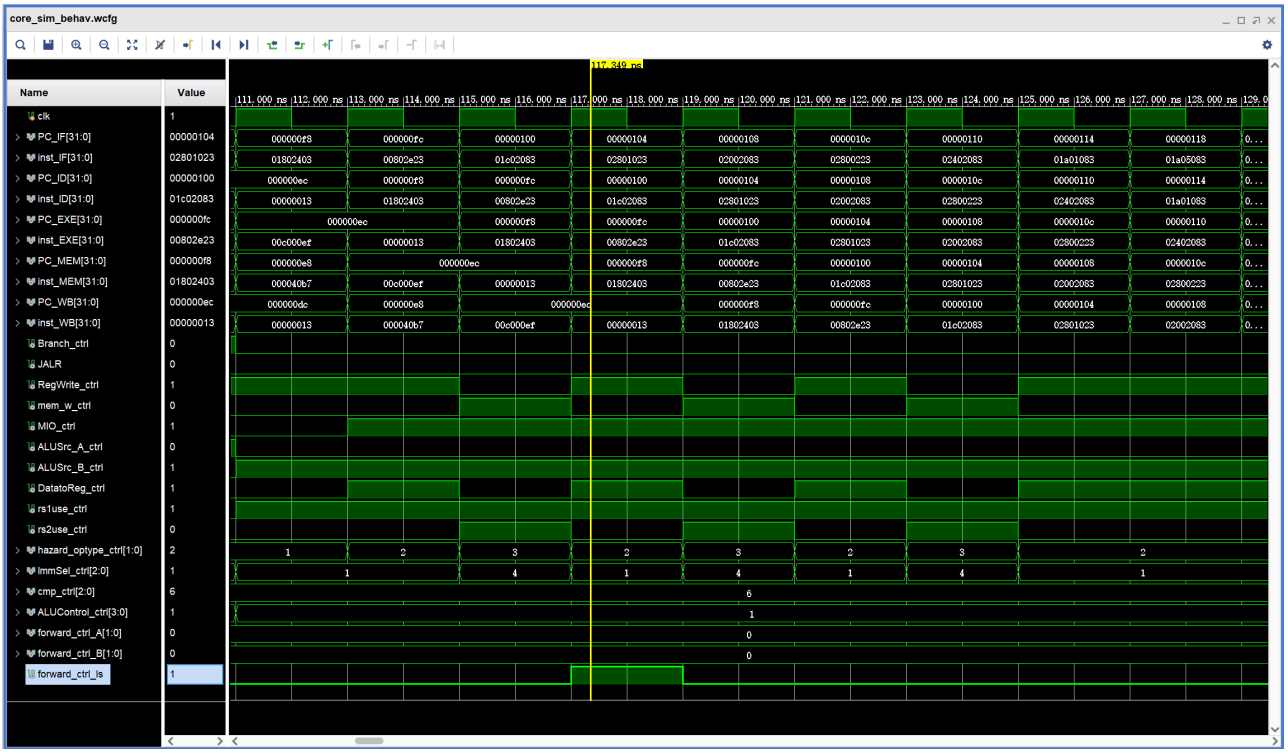
We analyze some of the typical cases when forwarding and stalling take effect.

First, for the simplest Registers Read-After-Write hazard, we can use the following example:

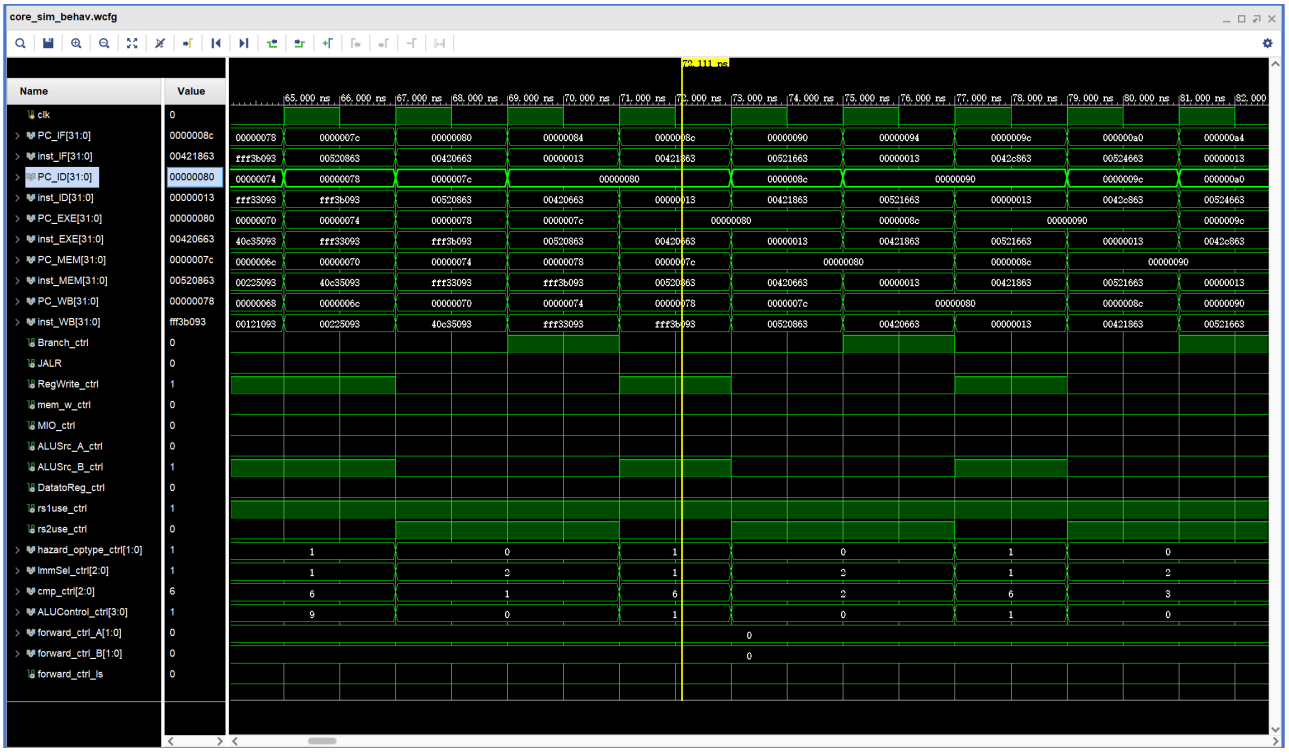


Here are the instructions: `lw x4, 8(x0), add x1, x2, x4  addi x1, x1, -1`, we can discover that the forwarding control signals are correctly activated and the data is correctly passed.

And this is also an example for load-use hazard, when forwarding is activated and stall is also needed, we can discover from the PC_IF that the instruction fetch is also stalled, which satisfies our design requirements.



Here is the load-store hazard, the instructions are: `lw x8, 24(x0)  sw x8, 28(x0)` . We can discover that the `forward_ctrl_ls` was activated in the EXE and MEM stage, and no stall was required.



Here is when the branch happens. the Instructions are: `beq x4, x5, label0  beq x4, x4, label0`.

We can discover that the second branch condition was satisfied, and the PC_ID stays the same because flush happens, flushing away the instruction that has already been fetched. In the previous instruction when the branch prediction failed, the instruction continues to flow.

5. Discussion and Conclusions

The experiment this time was aimed to implement a Pipeline CPU supporting stall, flush and forwarding.

We can tell that the increase of performance will require hardware overhead, and this is a tradeoff between design complexity and efficiency.

From the mistakes I've made, I think it's very important to make sure that all the digital signals must be correctly assigned or initialized, to make sure that there are no undefined situations, we should carefully design the verilog code.